# Unifying CEU's Course Information System

A Student-Centered Database Redesign — Final Presentation

David Diestre Rubio • Teodore Giorgobiani • Thanika Haltrich • Talha Sadiq

DNDS5020 – Data Management and Databases | Winter 2026 | March 2026

# Presentation Overview

## Problem & Solution

- Research problem recap
- Goals & hypothesis
- ER diagram (stable)
- GitHub overview

## Changes & Reflections

- Schema evolution
- Dummy vs. real data
- Data challenges
- Key queries demo
- Real-world feasibility

## Future Work

- What we'd add next
- What we'd do differently
- Key takeaways

# Part I: Problem & Solution

Recap of our stable project foundation

# The Research Problem

## Problem Statement

- Info across 3 platforms
- No unified access point
- Manual schedule building
- Old codes break search
- Cross-dept courses invisible

## Student Impact

- Platforms feel disconnected
- Course descriptions often missing
- Students miss eligible courses
- Manual schedule rebuilding
- Old codes make courses unfindable

# What We Want to Achieve

- *One unified database*
- *Replaces fragmented infrastructure*

## Unified View

- All departments, one place
- Cross-registrations included

## Smart Filtering

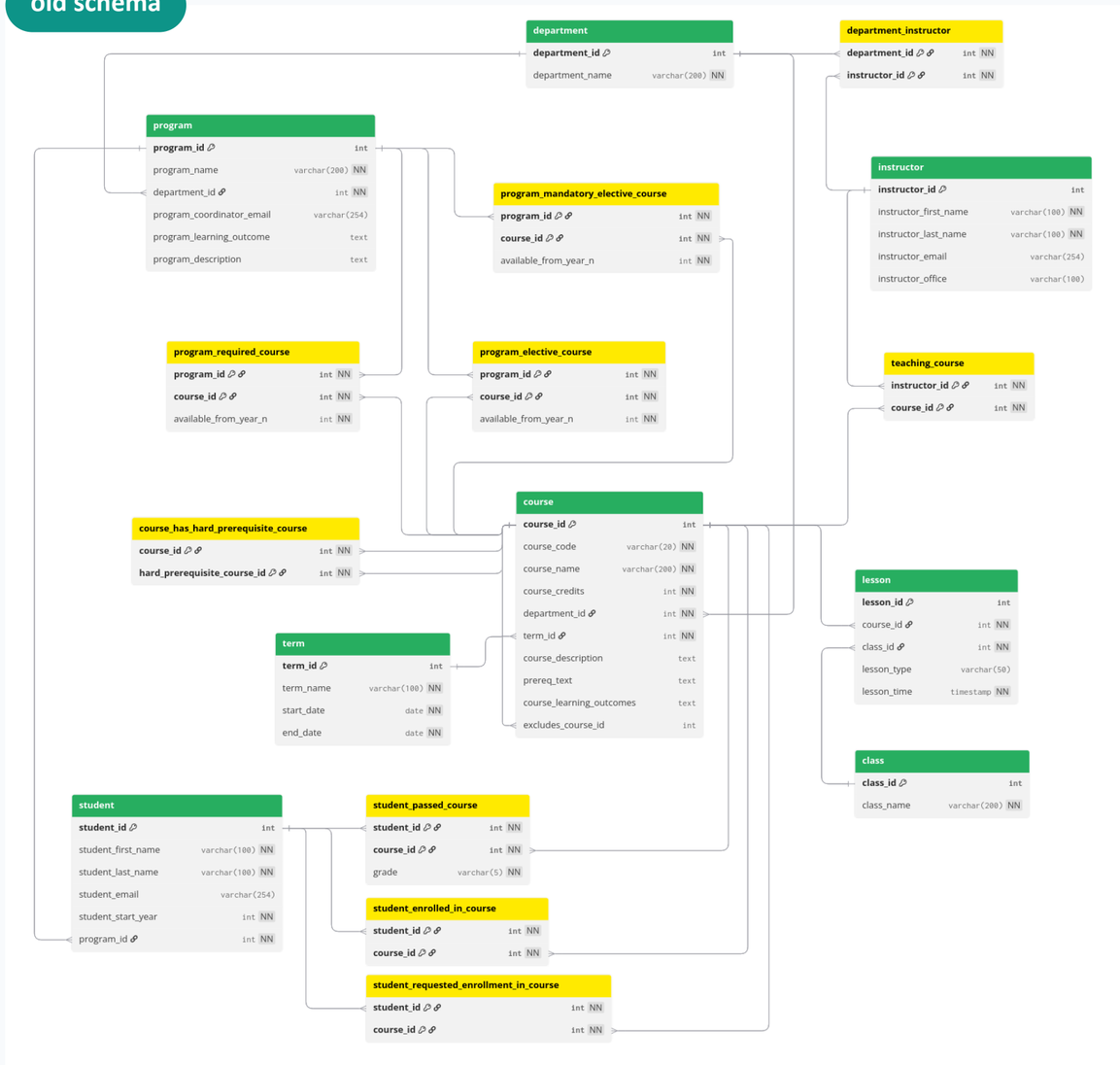
- Prerequisite-aware course view
- Filters passed/enrolled courses

## Better Planning

- Track credit progress
- Plan terms easily

# Database Design — old ER Diagram & changes

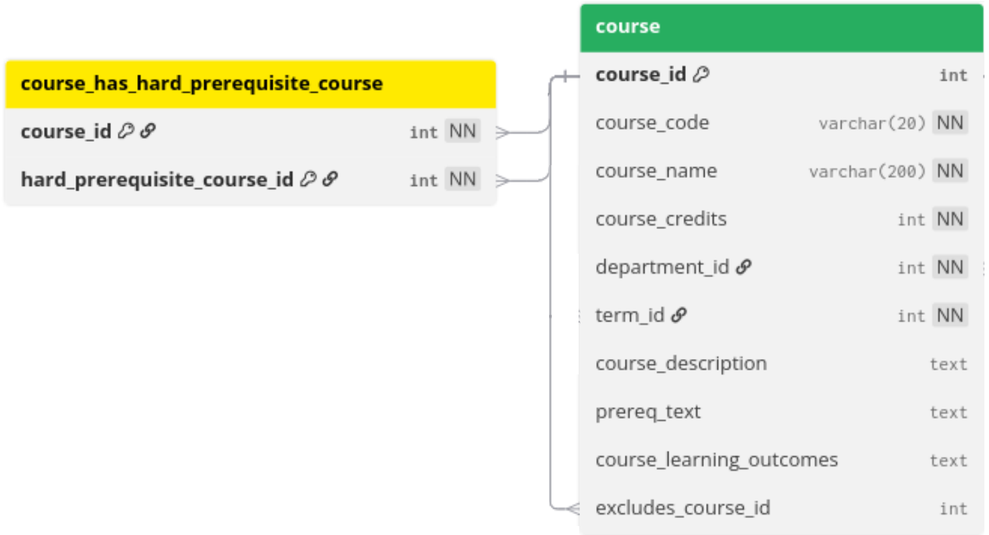
old schema



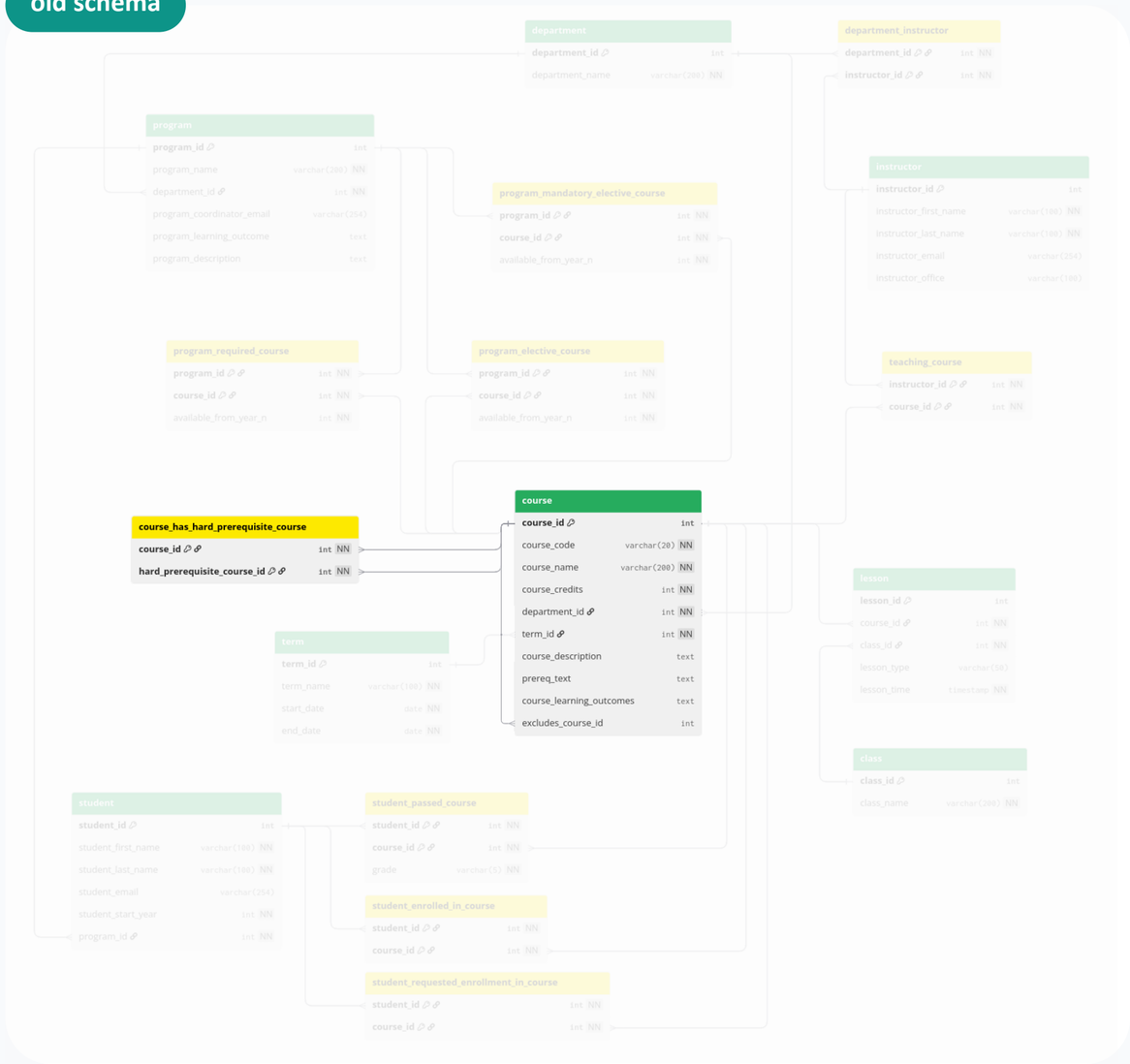
# Database Design — old ER Diagram & changes

## Core Entities

*course (old version)*



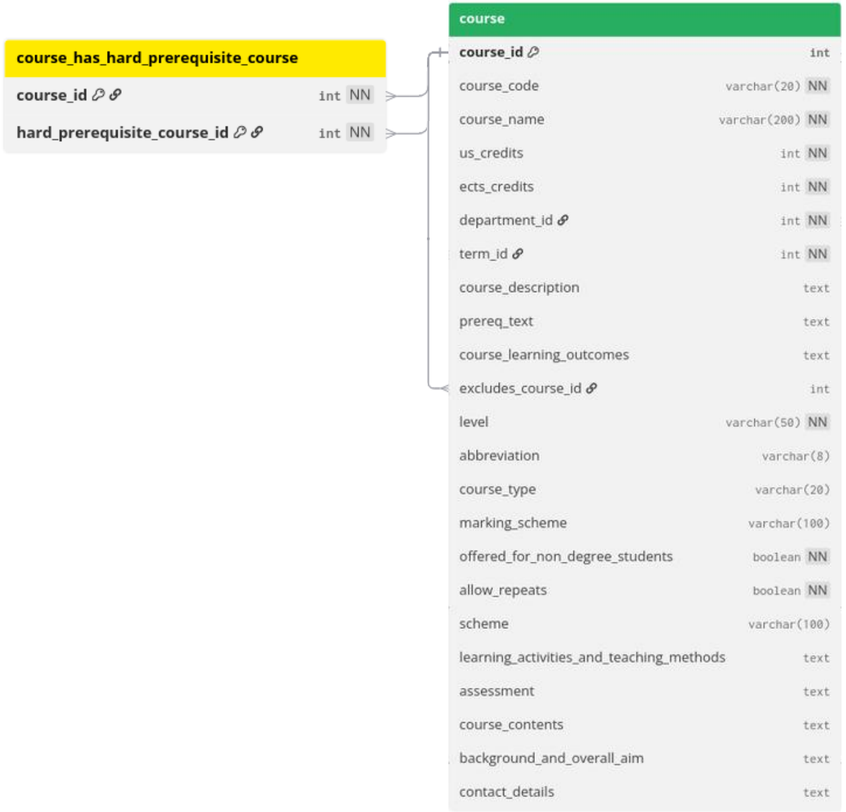

old schema



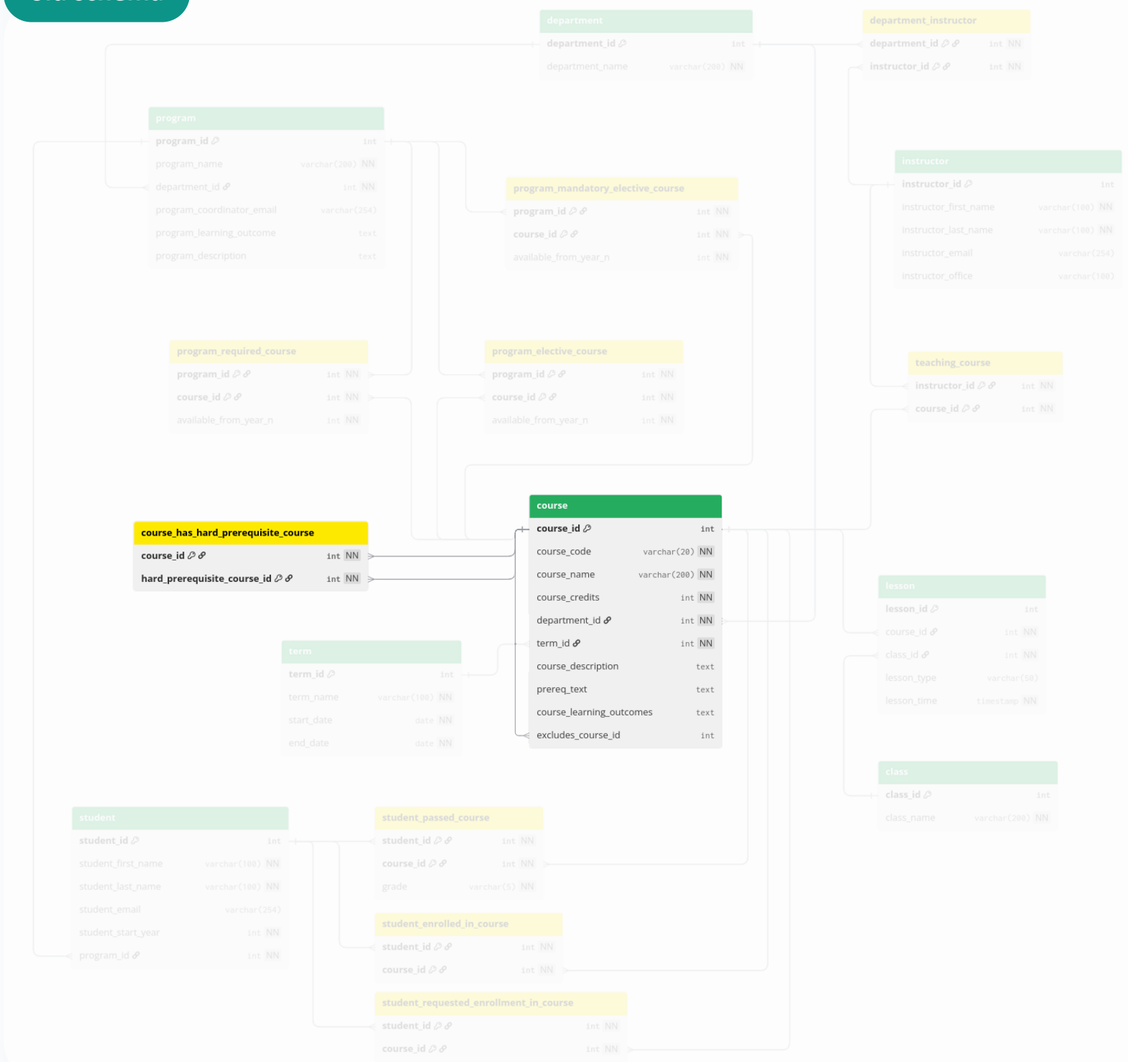
# Database Design — old ER Diagram & changes

## Core Entities

*course (actual version)*




old schema



# Database Design — old ER Diagram & changes

## Core Entities

*course*

***student (no changes)***

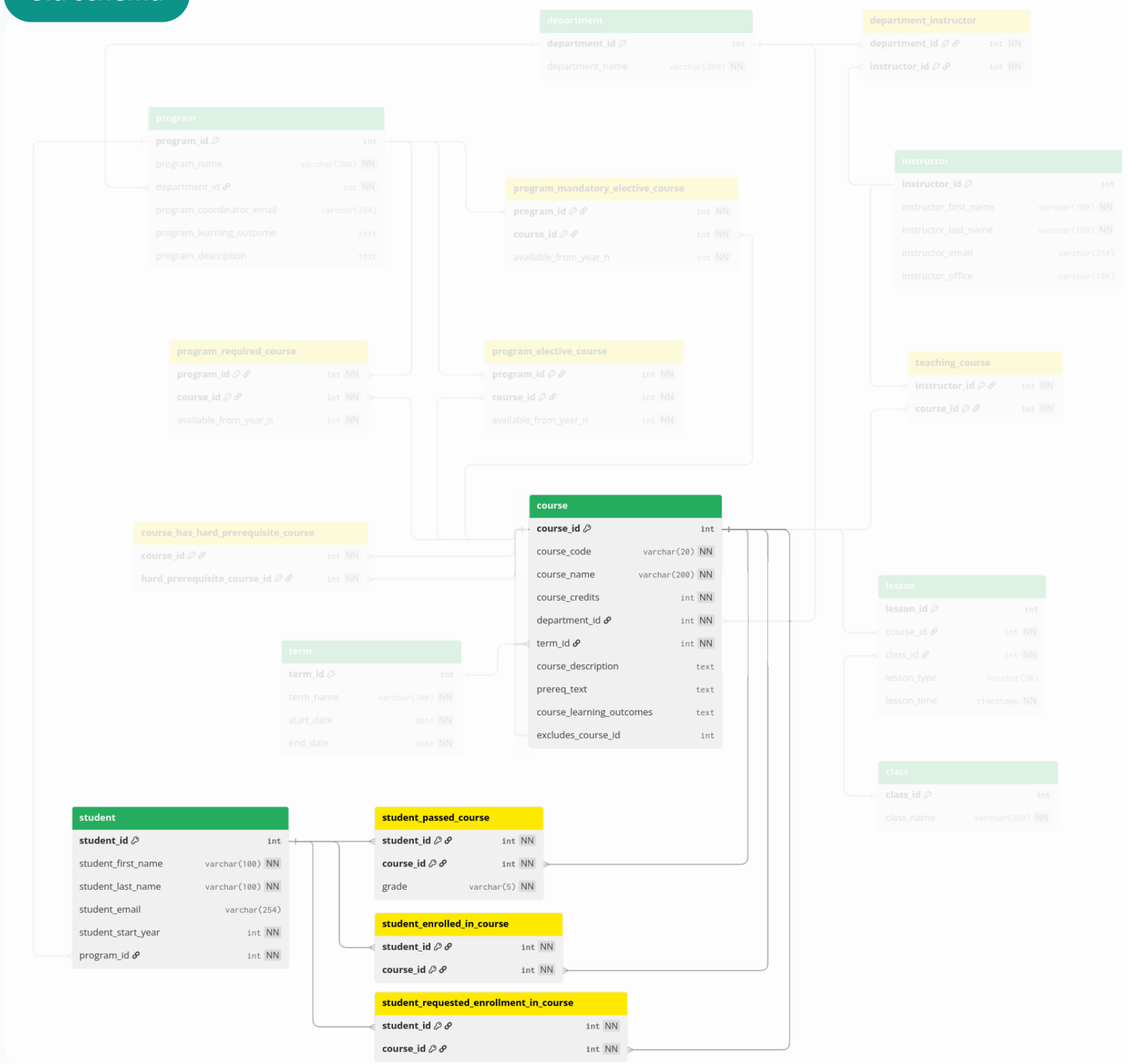
## Junction Tables (student - course)

*student_passed_course*

*student_enrolled_in_course*

*student_requested_enrollment_in_course*

old schema



# Database Design — old ER Diagram & changes

## Core Entities

*course*

*student*

***program (new attribute: corresponding_program_codes)***

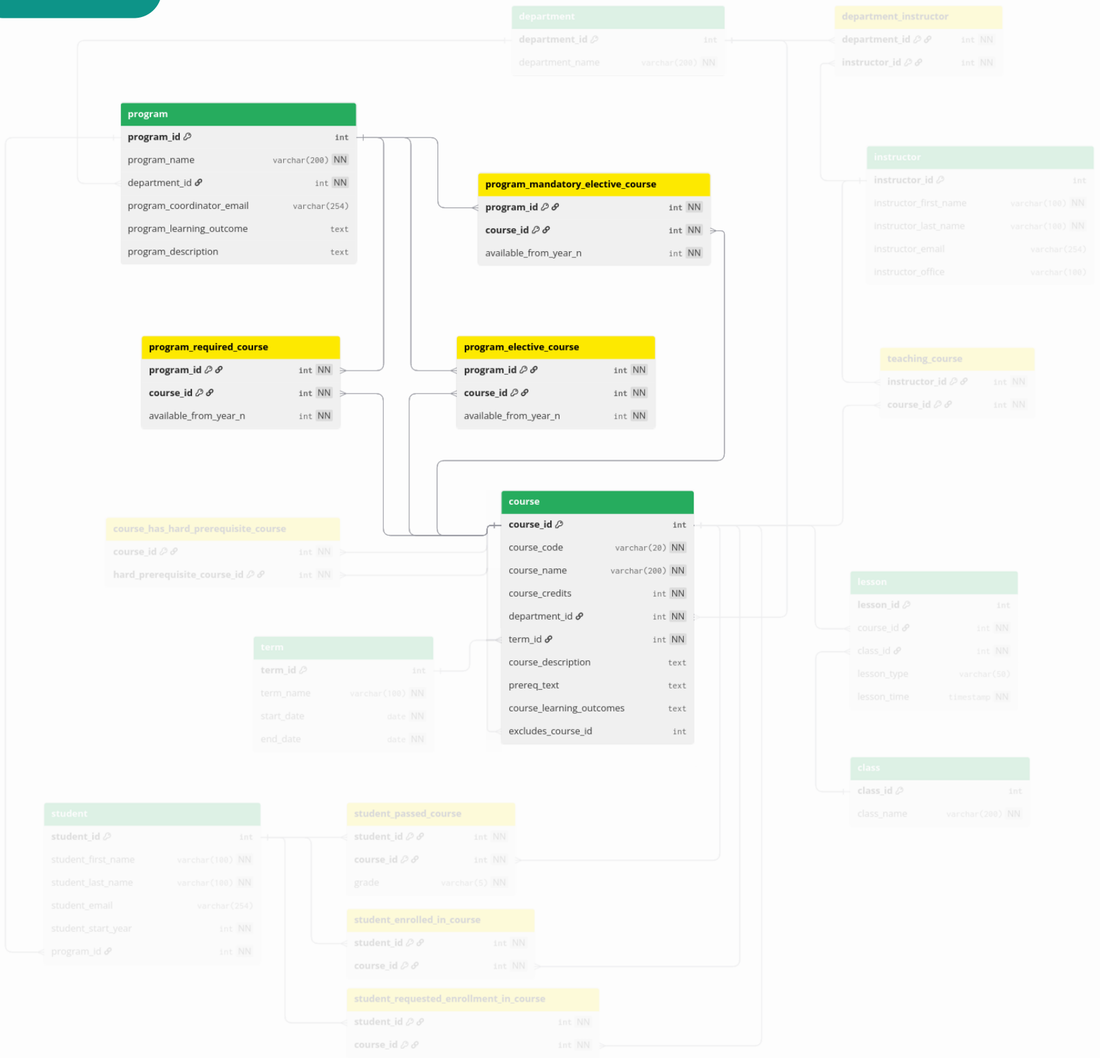
## Junction Tables (program - course)

*program_required_course ⟶ (renamed) program_mandatory_course*

*program_mandatory_elective_course*

*program_elective_course*

old schema



# Database Design — old ER Diagram & changes

## Other Entities

*department*

*instructor*

***lesson*** *(removed attribute* ***lesson_time*** *& added* ***start_time, end_time****)*

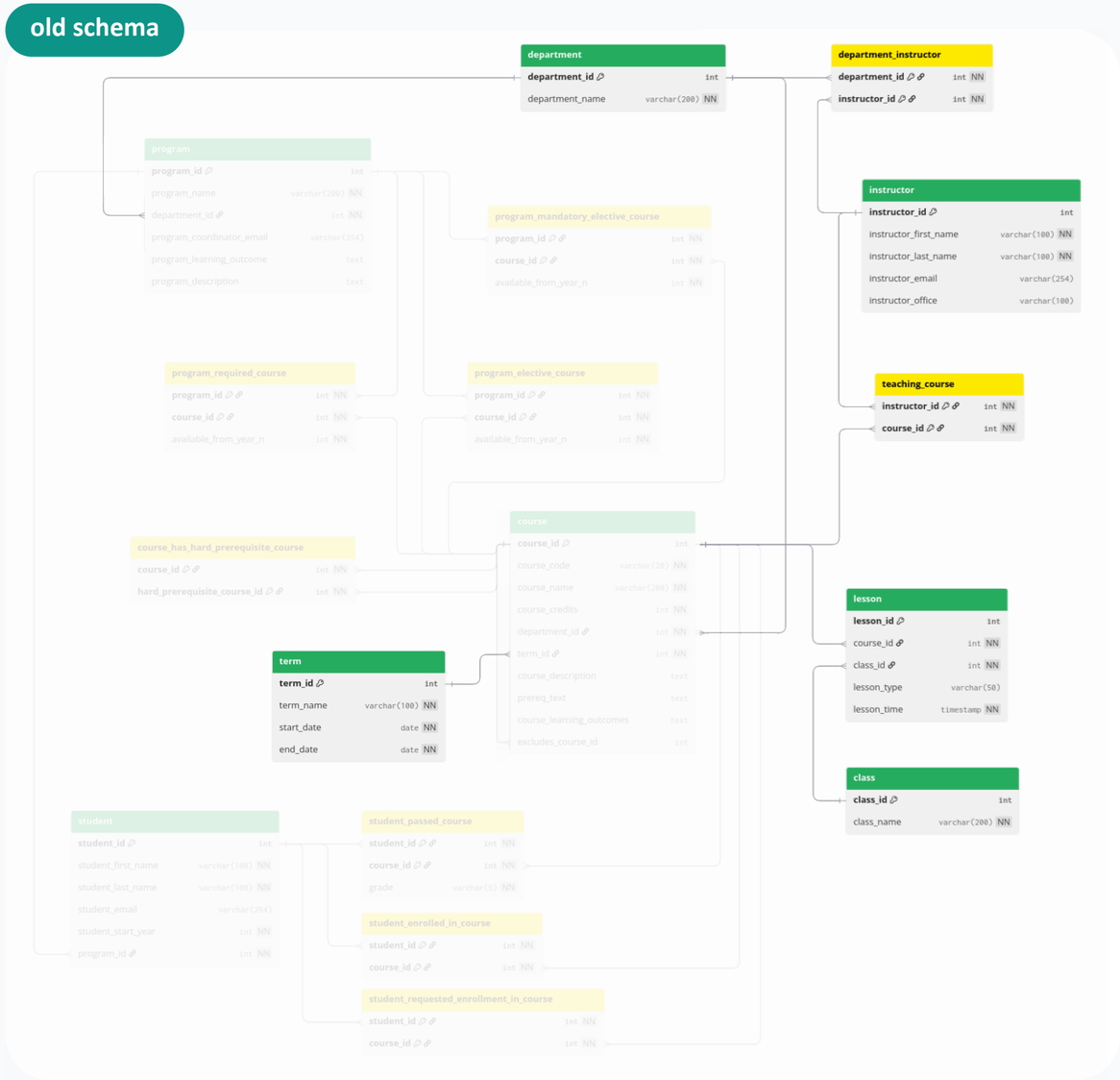
*term*

*class*

## Other Junction Tables

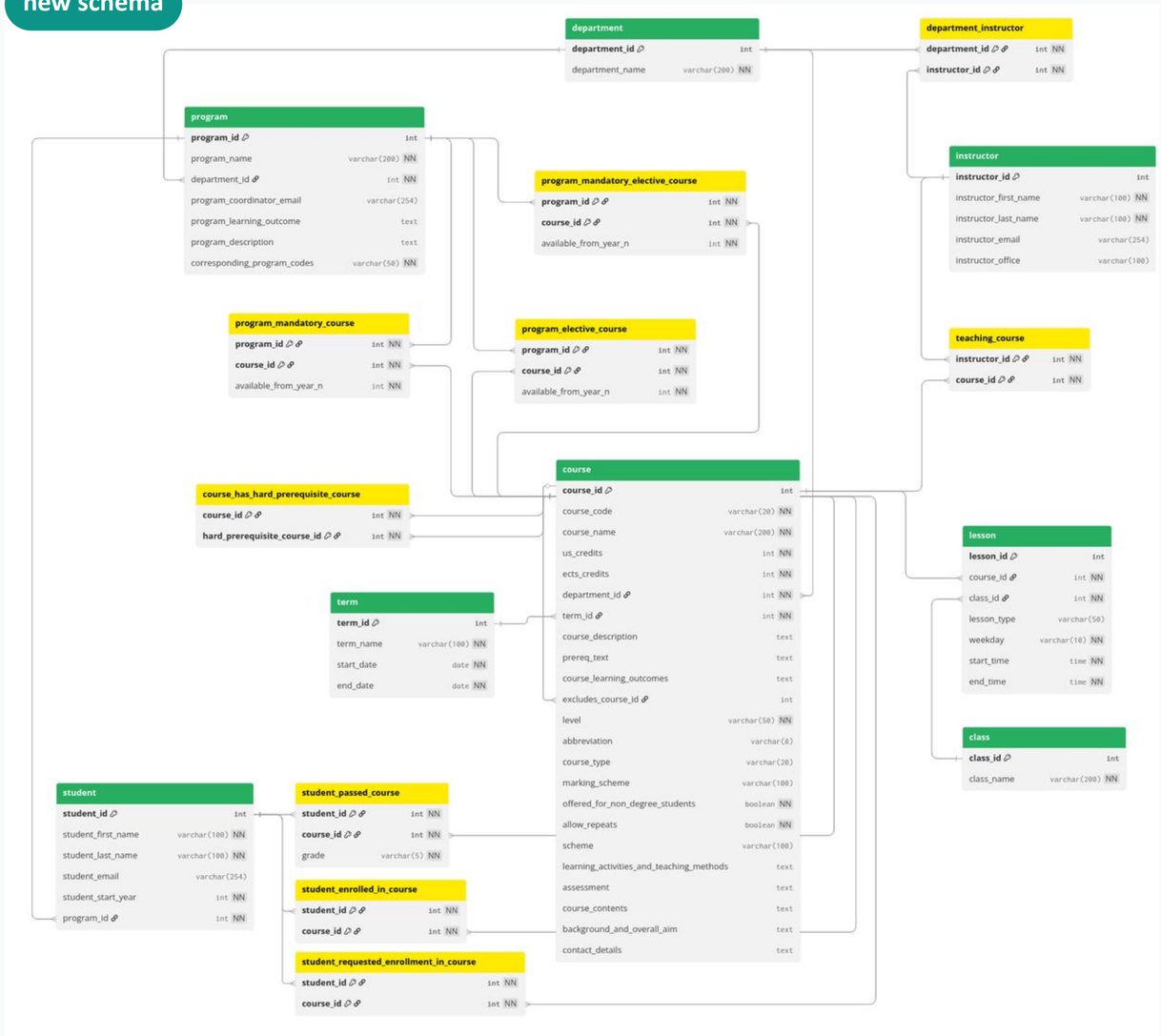
*department_instructor*

teaching_course



# Database Design — new ER Diagram

new schema



# GitHub Repository

```
CEU-DMDB-GroupH/
├ .gitattributes
├ .githooks/
├ .github/workflows/
├ .gitignore
├ .env
├ Makefile
├ docs/
│  ├ week_4_project_proposal/
│  ├ week_8_project_prototype/
│  └ week_12_project_presentation/
├ sql/
│  ├ 00_reset.sql
│  ├ 01_schema.sql
│  ├ 02_load.sql
│  ├ 03_views.sql
│  └ queries/
│     └ example_queries.sql
├ data_preprocessing/
└ data/
   ├ public/
   ├ private/
   └ dump_folder/
```

Preventing data leakage

## Example: psql command for 01_schema.sql

```
psql -v ON_ERROR_STOP=1 -v schema=$SCHEMA -f sql/01_schema.sql "host=$DB_HOST
port=$DB_PORT dbname=$DB_NAME user=$DB_USER password=$DB_PASSWORD"
```

## Make commands for easy setup

| | |
|---|---|
| 00_reset.sql | `make reset-truncate` / `make reset-drop` / `make reset-view` |
| 01_schema.sql | `make schema` |
| 02_load.sql | `make load` |
| 03_views.sql | `make views` |
| example_queries.sql | `make queries` |

# Part II: Changes & Reflections

What evolved, what we learned from real data

# From Dummy Data to Real Data

## Dummy Data

- Initially used fake data to get the big picture
  - useful when coming up with views and queries
- used as a model for implementing real data
- some fake data is still used where real data is unavailable

## Real CEU Data

- real data was significantly harder to implement
- had to update schema accordingly

# Real Data Challenges & Preprocessing

**Non-normal form data**
'Corresponding_programs' needed decomposition

**Messy data**
Text columns (such as course contents and learning outcomes) had commas as part of the text. Needed extra cleaning.

**Unavailable data (Program learning outcomes, student data)**
Generated fake data for these categories

# Real Data Challenges & Preprocessing

- **problems due to dirty data:**
  - 56% of the original data was empty; extensive data cleaning
  - Encoding/mojibake problem
- **problems when making tables:**
  - text fields having commas or quotations caused some fields to shift
  - Invalid value formats, mainly booleans and dates
  - invalid weekdays / schema mismatch
  - wrong CSV shape
- **adding constraints to the schema beforehand caused:**
  - Missing required values in non-null columns
  - Duplicate rows violating unique constraints

# Key Queries — Credit Calculation

## Mandatory credit progress per student

```sql
-- example: mandatory credit progress per student
SELECT s.student_last_name,
       p.program_name,
       SUM(c.course_credits)        AS total,
       COALESCE(SUM(c.course_credits) FILTER (WHERE spc.student_id
                IS NOT NULL), 0)                                  AS passed,
       SUM(c.course_credits) - COALESCE(SUM(c.course_credits)
           FILTER (WHERE spc.student_id IS NOT NULL), 0)          AS remaining
  FROM student s
  JOIN program p                   ON s.program_id  = p.program_id
  JOIN program_required_course prc ON p.program_id  = prc.program_id
  JOIN course c                    ON prc.course_id = c.course_id
  LEFT JOIN student_passed_course spc
         ON spc.student_id = s.student_id
        AND spc.course_id  = c.course_id
 GROUP BY s.student_id, p.program_name
 ORDER BY s.student_last_name;
```

# Key Queries — Credit Calculation

## Mandatory credit progress per student

```sql
-- example: mandatory credit progress per student
SELECT s.student_last_name,
       p.program_name,
       SUM(c.course_credits)        AS total,
       COALESCE(SUM(c.course_credits) FILTER (WHERE spc.student_id
                IS NOT NULL), 0)                                AS passed,
       SUM(c.course_credits) - COALESCE(SUM(c.course_credits)
           FILTER (WHERE spc.student_id IS NOT NULL), 0)        AS remaining
  FROM student s
  JOIN program p                   ON s.program_id  = p.program_id
  JOIN program_required_course prc ON p.program_id  = prc.program_id
  JOIN course c                    ON prc.course_id = c.course_id
  LEFT JOIN student_passed_course spc
         ON spc.student_id = s.student_id
        AND spc.course_id  = c.course_id
 GROUP BY s.student_id, p.program_name
 ORDER BY s.student_last_name;
```

| student_name | program_name | total_mandatory_credits | passed_mandatory_credits | remaining_mandatory |
|---|---|---|---|---|
| Ava Fischer | BSc Business Analytics | 5 | 0 | 5 |
| Emma Evans | BSc Business Analytics | 5 | 5 | 0 |
| Ethan Ionescu | BSc Computer Science | 23 | 6 | 17 |
| Liam Dobrev | MSc Data Science | 12 | 6 | 6 |
| Lucas Garcia | MSc Applied AI | 12 | 0 | 12 |
| Mia Anders | BSc Computer Science | 23 | 6 | 17 |
| Noah Balogh | BSc Computer Science | 23 | 11 | 12 |
| Olivia Chen | MSc Data Science | 12 | 6 | 6 |
| Sofia Horvath | MSc Applied AI | 12 | 0 | 12 |
| Zoe Jensen | MSc Data Science | 12 | 0 | 12 |

# Key Queries — Course Planning

## Available courses for a given student

```sql
-- example: available courses for a given student
SELECT c.course_code,
       c.course_name,
       'Required'   AS course_type
  FROM student s
  JOIN program_required_course prc USING (program_id)
  JOIN course c                   USING (course_id)
 WHERE s.student_id = 2
   AND c.course_id NOT IN (SELECT course_id
                           FROM student_passed_course
                          WHERE student_id = s.student_id)
 ORDER BY course_type, course_code;
-- (UNION ALL with elective & mandatory-elective omitted for brevity)
```

# Key Queries — Course Planning

## Available courses for a given student

```sql
-- example: available courses for a given student
SELECT c.course_code,
       c.course_name,
       'Required'   AS course_type
  FROM student s
  JOIN program_required_course prc USING (program_id)
  JOIN course c                   USING (course_id)
 WHERE s.student_id = 2
   AND c.course_id NOT IN (SELECT course_id
                             FROM student_passed_course
                            WHERE student_id = s.student_id)
 ORDER BY course_type, course_code;
-- (UNION ALL with elective & mandatory-elective omitted for brevity)
```

| course_code | course_name | course_credits | term_name | course_type | available_from_year_n |
|---|---|---|---|---|---|
| BA201 | Business Intelligence | 5 | Spring 2026 | Elective | 3 |
| DS101 | Statistics for Data Science | 6 | Fall 2025 | Elective | 2 |
| BA201 | Business Intelligence | 5 | Spring 2026 | Mandatory Elective | 2 |
| CS301 | Database Systems | 6 | Summer 2026 | Required | 3 |

# Key Queries — Enrollment Logic

**Auto-approve or flag pending enrollment requests**

**real life example: enrolling in the course ‘Numerical methods in calculus and algebra’**

| | course_name | hard_prerequisite_course_id | course_name |
|---|---|---|---|
| 1 | Numerical Methods in Calculus and Algebra | 474 | Introduction to Statistics |
| 2 | Numerical Methods in Calculus and Algebra | 472 | Introduction to Programming in Python |
| 3 | Numerical Methods in Calculus and Algebra | 568 | Introduction to Linear Algebra |
| 4 | Numerical Methods in Calculus and Algebra | 569 | Multivariate Calculus |

# Key Queries — Enrollment Logic

**Auto-approve or flag pending enrollment requests**

**real life example: enrolling in the course ‘Numerical methods in calculus and algebra’**

| | course_name | hard_prerequisite_course_id | course_name |
|---|---|---|---|
| 1 | Numerical Methods in Calculus and Algebra | 474 | Introduction to Statistics |
| 2 | Numerical Methods in Calculus and Algebra | 472 | Introduction to Programming in Python |
| 3 | Numerical Methods in Calculus and Algebra | 568 | Introduction to Linear Algebra |
| 4 | Numerical Methods in Calculus and Algebra | 569 | Multivariate Calculus |

| | student_id | student_first_name | student_last_name | student_email | student_start_year | program_id |
|---|---|---|---|---|---|---|
| 1 | 201 | Mia | Corduneanu-Huci | corduneanu-huci_mia@student.ceu.ed | 2,024 | 3 |

# Key Queries — Enrollment Logic

**Auto-approve or flag pending enrollment requests**

**real life example: enrolling in the course ‘Numerical methods in calculus and algebra’**

| | course_name | hard_prerequisite_course_id | course_name |
|---|---|---|---|
| 1 | Numerical Methods in Calculus and Algebra | 474 | Introduction to Statistics |
| 2 | Numerical Methods in Calculus and Algebra | 472 | Introduction to Programming in Python |
| 3 | Numerical Methods in Calculus and Algebra | 568 | Introduction to Linear Algebra |
| 4 | Numerical Methods in Calculus and Algebra | 569 | Multivariate Calculus |

| | student_id | student_first_name | student_last_name | student_email | student_start_year | program_id |
|---|---|---|---|---|---|---|
| 1 | 201 | Mia | Corduneanu-Huci | corduneanu-huci_mia@student.ceu.ed | 2,024 | 3 |

| | | | |
|---|---|---|---|
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Linear Algebra |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Programming in Python |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Statistics |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Communities, Societies, States: Foundations of Social Theory |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Calculus |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Algorithms and Data Structures |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Linear Algebra |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Programming in Python |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Statistics |
| Mia | Corduneanu-Huci | Multivariate Calculus | Communities, Societies, States: Foundations of Social Theory |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Calculus |
| Mia | Corduneanu-Huci | Multivariate Calculus | Algorithms and Data Structures |

# Key Queries — Enrollment Logic

**Auto-approve or flag pending enrollment requests**

**real life example: enrolling in the course ‘Numerical methods in calculus and algebra’**

| | course_name | hard_prerequisite_course_id | course_name |
|---|---|---|---|
| 1 | Numerical Methods in Calculus and Algebra | 474 | Introduction to Statistics |
| 2 | Numerical Methods in Calculus and Algebra | 472 | Introduction to Programming in Python |
| 3 | Numerical Methods in Calculus and Algebra | 568 | Introduction to Linear Algebra |
| 4 | Numerical Methods in Calculus and Algebra | 569 | Multivariate Calculus |

| | student_id | student_first_name | student_last_name | student_email | student_start_year | program_id |
|---|---|---|---|---|---|---|
| 1 | 201 | Mia | Corduneanu-Huci | corduneanu-huci_mia@student.ceu.ed | 2,024 | 3 |

| | | | |
|---|---|---|---|
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Linear Algebra |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Programming in Python |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Statistics |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Communities, Societies, States: Foundations of Social Theory |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Introduction to Calculus |
| Mia | Corduneanu-Huci | Introduction to Econometrics | Algorithms and Data Structures |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Linear Algebra |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Programming in Python |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Statistics |
| Mia | Corduneanu-Huci | Multivariate Calculus | Communities, Societies, States: Foundations of Social Theory |
| Mia | Corduneanu-Huci | Multivariate Calculus | Introduction to Calculus |
| Mia | Corduneanu-Huci | Multivariate Calculus | Algorithms and Data Structures |

# Key Queries — Enrollment Logic

**Auto-approve or flag pending enrollment requests**

**real life example: enrolling in the course ‘Numerical methods in calculus and algebra’**

| | course_name | hard_prerequisite_course_id | course_name |
|---|---|---|---|
| 1 | Numerical Methods in Calculus and Algebra | 474 | Introduction to Statistics |
| 2 | Numerical Methods in Calculus and Algebra | 472 | Introduction to Programming in Python |
| 3 | Numerical Methods in Calculus and Algebra | 568 | Introduction to Linear Algebra |
| 4 | Numerical Methods in Calculus and Algebra | 569 | Multivariate Calculus |

| student_id | student_first_name | student_last_name | student_email | student_start_year | program_id |
|---|---|---|---|---|---|
| 202 | Juan | Roset-Moreno | roset-moreno_juan@student.ceu.edu | 2,025 | 3 |

| student_first_name | student_last_name | enrolled_course_name | passed_course_name |
|---|---|---|---|

# Key Queries — Enrollment Logic

## Auto-approve or flag pending enrollment requests

```sql
SELECT s.student_first_name || ' ' || s.student_last_name AS student_name,
       c.course_code,
       c.course_name,
       COALESCE(
         STRING_AGG(prereq.course_name, ', ' ORDER BY prereq.course_name),
         'None'
       ) AS hard_prerequisites, --self join on course to get hard prereqs
       c.prereq_text       AS soft_prerequisites,
       CASE
           WHEN COUNT(chpc.hard_prerequisite_course_id) = 0
                THEN 'AUTO-APPROVE'
           WHEN COUNT(spc.course_id)+COUNT(seic.course_id) = COUNT(chpc.hard_prerequisite_course_id)
                THEN 'AUTO-APPROVE'
           ELSE 'FLAG: prerequisites not met'
       END AS decision
FROM student_requested_enrollment_in_course req
JOIN student s      ON req.student_id = s.student_id
JOIN course  c      ON req.course_id  = c.course_id
LEFT JOIN course_has_hard_prerequisite_course chpc
       ON chpc.course_id = c.course_id
```

```sql
LEFT JOIN course prereq
       ON prereq.course_id = chpc.hard_prerequisite_course_id
LEFT JOIN student_passed_course spc
       ON spc.student_id = req.student_id
      AND spc.course_id  = chpc.hard_prerequisite_course_id
LEFT JOIN student_enrolled_in_course seic
       ON seic.student_id = req.student_id
      AND seic.course_id  = chpc.hard_prerequisite_course_id
GROUP BY req.student_id,
         req.course_id,
         s.student_id,
         s.student_first_name,
         s.student_last_name,
         c.course_code,
         c.course_name,
         c.prereq_text
HAVING s.student_id > 200
ORDER BY decision, s.student_last_name
LIMIT 5;
```

# Key Queries — Enrollment Logic

**Auto-approve or flag pending enrollment requests**

| | course_name | hard_prerequisites | soft_prerequisites | decision |
|---|---|---|---|---|
| 1 | Numerical Methods in Calculus and Algebra | Introduction to Linear Algebra, Introduction to Programming in | Prerequisites: Introduction to Linear Algebra, Multivariate Calc | AUTO-APPROVE |
| 2 | Numerical Methods in Calculus and Algebra | Introduction to Linear Algebra, Introduction to Programming in | Prerequisites: Introduction to Linear Algebra, Multivariate Calc | FLAG: prerequisites not met |

# Key Queries — Rescheduling

```sql
with params as (
    select 1::int as course_id
),
missed_course as (
    select
        c.course_id,
        c.course_name
    from course c
    join params p
        on c.course_id = p.course_id
),
affected_students as (
    select distinct
        sec.student_id
    from student_enrolled_in_course sec
    join params p
        on sec.course_id = p.course_id
),
missed_instructor as (
    select distinct
        tc.instructor_id
    from teaching_course tc
    join params p
        on tc.course_id = p.course_id
),
candidate_slots as (
    select 'monday' as day, time '08:20' as start_time, time '09:50' as end_time
    union all
    select 'monday', time '10:00', time '11:30'
    union all
    select 'monday', time '11:40', time '13:10'
    union all
    select 'tuesday', time '08:20', time '09:50'
    union all
    select 'tuesday', time '10:00', time '11:30'
    union all
    select 'wednesday', time '08:20', time '09:50'
    union all
    select 'wednesday', time '10:00', time '11:30'
    union all
    select 'thursday', time '13:30', time '15:00'
    union all
    select 'friday', time '10:00', time '11:30'
),
```

```sql
student_conflicts as (
    select distinct
        cs.day,
        cs.start_time,
        cs.end_time
    from candidate_slots cs
    join affected_students s
        on 1 = 1
    join student_enrolled_in_course sec
        on sec.student_id = s.student_id
    join lesson l
        on l.course_id = sec.course_id
    join params p
        on 1 = 1
    where sec.course_id <> p.course_id
      and lower(l.weekday) = cs.day
      and l.start_time < cs.end_time
      and l.end_time > cs.start_time
),
instructor_conflicts as (
    select distinct
        cs.day,
        cs.start_time,
        cs.end_time
    from candidate_slots cs
    join missed_instructor mi
        on 1 = 1
    join teaching_course tc
        on tc.instructor_id = mi.instructor_id
    join lesson l
        on l.course_id = tc.course_id
    join params p
        on 1 = 1
    where tc.course_id <> p.course_id
      and lower(l.weekday) = cs.day
      and l.start_time < cs.end_time
      and l.end_time > cs.start_time
),
```

# Key Queries Rescheduling

```sql
available_rooms as (
    select
        cs.day,
        cs.start_time,
        cs.end_time,
        string_agg(cl.class_name, ', ' order by cl.class_name) as rooms_available
    from candidate_slots cs
    join class cl
        on 1 = 1
    where not exists (
        select 1
        from lesson l
        where l.class_id = cl.class_id
          and lower(l.weekday) = cs.day
          and l.start_time < cs.end_time
          and l.end_time > cs.start_time
    )
    group by cs.day, cs.start_time, cs.end_time
)
select
    mc.course_name,
    cs.day,
    cs.start_time,
    cs.end_time,
    ar.rooms_available
from candidate_slots cs
join missed_course mc
    on 1 = 1
left join student_conflicts sc
    on cs.day = sc.day
   and cs.start_time = sc.start_time
   and cs.end_time = sc.end_time
left join instructor_conflicts ic
    on cs.day = ic.day
   and cs.start_time = ic.start_time
   and cs.end_time = ic.end_time
left join available_rooms ar
    on cs.day = ar.day
   and cs.start_time = ar.start_time
   and cs.end_time = ar.end_time
where sc.day is null
  and ic.day is null
order by
    case cs.day
        when 'monday' then 1
        when 'tuesday' then 2
        when 'wednesday' then 3
        when 'thursday' then 4
        when 'friday' then 5
        else 6
    end,
    cs.start_time;
```

| course_name | day | start_time | end_time | rooms_available |
|---|---|---|---|---|
| Academic Writing for Historical Studies | wednesday | 08:20:00 | 09:50:00 | N13 118 Tiered, N13 307/A, QS A-101, QS A-103, QS A-201, QS A-215, QS A-415, QS A-419, QS A-516 |
| Academic Writing for Historical Studies | wednesday | 10:00:00 | 11:30:00 | N13 118 Tiered, N13 307/A, QS A-201, QS A-415, QS A-419, QS A-516, QS B-216, QS B-319 Senate, Q |
| Academic Writing for Historical Studies | thursday | 13:30:00 | 15:00:00 | N13 301, N13 517, QS A-019 Comp Lab, QS A-103, QS A-201, QS A-214, QS A-215, QS A-415, QS B-2 |
| Academic Writing for Historical Studies | friday | 10:00:00 | 11:30:00 | N13 301, N13 517, QS A-019 Comp Lab, QS A-102, QS A-211, QS A-214, QS A-419, QS A-516, QS B-4 |

# View — Student timetable

```
create materialized view student_timetable as ...
```

| weekday | course | time | lesson_type | room |
|---|---|---|---|---|
| monday | INTR5064 - International Political Economy of Digital Everything | 12:45:00 - 14:30:00 | seminar | QS D-103 |
| tuesday | DOPP5305 - Mobilizing Contention? Social Movements as Demo | 17:40:00 - 20:25:00 | lecture | QS A-516 |
| wednesday | DOPP5671 - Tackling Contemporary Policy Issues: Seminar | 19:30:00 - 21:10:00 | lecture | QS A-211 |
| friday | UGST4188 - Fundamentals of Documentary Photography | 12:30:00 - 13:45:00 | seminar | QS D-108 |

# Could This Work at a Real University?

## What Works

- *Unified view saves student time*
- *Prerequisite logic prevents errors*
- *Credit tracking automates advising*
- *Teachers can use it for room availability / course signup requests*
- *students can use it for timetable, enrolment or credit calculations and schedules.*

## Real-World Constraints

- *CEU is tied to a third-party data management contract.*
- *Legacy systems are hard to replace, even when data exists*
- *Data quality and availability are limited; for example, schedules appear only about two weeks before term.*
- *The process depends heavily on administration, so much is outside our control.*
- *Similar programs with different names make course mapping inconsistent.*

# Part III: Future Work

What comes next — and what we'd do differently

# What We Would Add Next

## Student-Facing UI

*A web frontend where students can search, filter, and plan courses in one place*
*Hopefully with schedules that include all courses taken by the student, with correct timings.*

## TimeEdit Integration

*API connection to pull live scheduling data instead of relying on fake data*
*Also pull live courses data from there instead of relying on the data file sent by Petra. (some courses get dropped)*

## Role-Based Access

*Different views for students, program directors, and administrators*
*Allowing program directors and administrators ability to change and update data (add learning outcomes for insta*

## Conflict Detection

*Automated schedule conflict checking and smart course recommendations*
*Also, automated conflict checking on what courses a student can take depending on their year.*

# What We'd Do Differently

*If we started from scratch...*

**Start with real data from day one**
Design the schema around actual data quality and availability, not idealised assumptions

**Engage stakeholders earlier**
We dropped the CEU collaboration goal — should have scoped it as academic prototype from the start

**Validate schema with queries immediately**
Writing test queries early would have caught design issues before they became structural

**Have a Data Management plan from the start**
Managing private data needs strong initial conventions to avoid issues and have smoother collaborative work

**Clean Department names manually**
Inconsistent names for the same program (DSS vs QSS) makes the db less efficient.

# What We'd Do Differently

*If we started from scratch...*

**Database is highly dependant on the data we have**
Missing data and irregular data decrease the potential efficiency of a DB.

**Start data cleaning earlier**
To ensure we populate the database with high quality data.

# References


[1] E. Lvov and T. Haltrich. CEU Students' experience with course selection process [Survey]. 2025.
[2] Universitat Rovira i Virgili. URV Intranet. 2026. https://intranet.urv.cat/
[3] Holistics Software. dbdiagram.io. 2026. https://dbdiagram.io/
[4] Petra Karl Novak. Suggestion for Learning Outcomes attributes. 2026.
[5] Central European University. Course Data [Data set]. 2025.
[6] Central European University. TimeEdit Course Data [Data set]. 2026.
[7] T. Giorgobiani. CEU Course Management Experience [Survey]. 2026.